

```
"""
```

```
Bot de surveillance des logements CROUS.
```

```
Vérifie une page de résultats de recherche du site trouverunlogement.lescrous.fr  
et envoie une alerte Telegram pour chaque NOUVEAU logement détecté.
```

```
Configuration via variables d'environnement :
```

```
- CROUS_SEARCH_URL : l'URL de recherche (voir README pour l'obtenir)
```

```
- TELEGRAM_BOT_TOKEN : le token de ton bot Telegram
```

```
- TELEGRAM_CHAT_ID : l'ID de la conversation où envoyer les alertes
```

```
"""
```

```
import os
```

```
import sys
```

```
import json
```

```
import requests
```

```
from bs4 import BeautifulSoup
```

```
STATE_FILE = "seen_ids.json"
```

```
def load_seen_ids():
```

```
    if os.path.exists(STATE_FILE):
```

```
        with open(STATE_FILE, "r", encoding="utf-8") as f:
```

```
            return set(json.load(f))
```

```
    return set()
```

```
def save_seen_ids(ids):
```

```
    with open(STATE_FILE, "w", encoding="utf-8") as f:
```

```
        json.dump(sorted(ids), f, ensure_ascii=False, indent=2)
```

```
def fetch_listings(url):
```

```
    headers = {
```

```
        "User-Agent": "Mozilla/5.0 (compatible; CrousWatcher/1.0; +personal use)"
```

```
    }
```

```
    resp = requests.get(url, headers=headers, timeout=30)
```

```
    resp.raise_for_status()
```

```
    soup = BeautifulSoup(resp.text, "html.parser")
```

```
    listings = []
```

```
    # Chaque logement est un <li> contenant un lien vers /tools/{id}/accommodatio
```

```
    for link in soup.select('a[href*="/accommodations/"]'):
```

```
        href = link.get("href", "")
```

```

        if "/accommodations/" not in href:
            continue
        acc_id = href.rstrip("/").split("/")[-1]
        if not acc_id.isdigit():
            continue

        # Remonter au bloc <li> parent pour récupérer les infos autour du lien
        block = link.find_parent("li")
        text = block.get_text(" ", strip=True) if block else link.get_text(" ", s

    listings.append({
        "id": acc_id,
        "name": link.get_text(strip=True),
        "url": "https://trouverunlogement.lescrous.fr" + href if href.startswith
        "details": text,
    })

# Dédupliquer par id (le lien apparaît parfois deux fois : image + titre)
unique = {}
for item in listings:
    unique[item["id"]] = item
return list(unique.values())

def send_telegram_message(token, chat_id, text):
    url = f"https://api.telegram.org/bot{token}/sendMessage"
    resp = requests.post(url, data={
        "chat_id": chat_id,
        "text": text,
        "parse_mode": "HTML",
        "disable_web_page_preview": False,
    }, timeout=30)
    resp.raise_for_status()

def main():
    search_url = os.environ.get("CROUS_SEARCH_URL")
    bot_token = os.environ.get("TELEGRAM_BOT_TOKEN")
    chat_id = os.environ.get("TELEGRAM_CHAT_ID")

    if not all([search_url, bot_token, chat_id]):
        print("ERREUR : il manque une variable d'environnement (CROUS_SEARCH_URL,
              TELEGRAM_BOT_TOKEN ou TELEGRAM_CHAT_ID).")
        sys.exit(1)

    seen_ids = load_seen_ids()
    listings = fetch_listings(search_url)

```

```

current_ids = {item["id"] for item in listings}

new_items = [item for item in listings if item["id"] not in seen_ids]

if new_items:
    print(f"{len(new_items)} nouveau(x) logement(s) détecté(s).")
    for item in new_items:
        message = (
            f"🏠 <b>Nouveau logement CROUS disponible !</b>\n\n"
            f"<b>{item['name']}</b>\n"
            f"{item['details']}\n\n"
            f"{item['url']}"
        )
        send_telegram_message(bot_token, chat_id, message)
    else:
        print("Aucun nouveau logement pour cette vérification.")

# On ne garde que les ids toujours présents + les nouveaux,
# pour "oublier" les logements qui ont disparu (repris) et pouvoir
# re-alerter s'ils reviennent plus tard.
save_seen_ids(current_ids)

if __name__ == "__main__":
    main()

```